

7.4. AJAX (Asynchronous Javascript And XML)

Es un conjunto de **técnicas** de desarrollo web que **permiten que las aplicaciones web funcionen de forma asíncrona**, procesando cualquier solicitud al servidor en segundo plano.

El comportamiento de forma asíncrona, como ya hemos visto en el anterior apartado, nos permite poder mostrar nuevos datos enviados por el servidor sin tener que recargar la página, que continuará disponible mientras se reciben y procesan los datos enviados por el servidor en segundo plano.

7.4.1. Asíncronía

JavaScript ofrece varios mecanismos para **gestionar la asíncronía**:

- Mediante **callbacks**: Forma más clásica de gestionar la asíncronía.
- Mediante **promesas y 'thennables'**: Mecanismo moderno para gestionar la asíncronía de forma no bloqueante.
- Mediante **async/await**: Forma clara y fácil de leer de manejar promesas. La más usada hoy en día.

- Mediante **top-level await**: Variación de la anterior, útil en algunos contextos de módulos modernos, pero su uso está más limitado (no en todos los entornos, y no siempre necesario).

<https://lenguajejs.com/asincronia/async-await/top-level-await/>

7.4.1.1. Callbacks

Las funciones callback (funciones de retrollamada) no son más que un tipo de **funciones que se pasan por parámetro a otras funciones**. Además, los parámetros de esos callbacks toman un valor especial en el interior de la función.

```
function obtenerDatos(callback) {  
  console.log("Solicitando datos...");  
  
  setTimeout(() => {  
    const datos = { nombre: "Ana", edad: 25 };  
    callback(datos);    // ← aquí llamamos al callback cuando termina  
  }, 2000);  
}  
  
// Uso de la función  
obtenerDatos(function(resultado) {  
  console.log("Datos recibidos:", resultado);  
});  
  
console.log("Yo me ejecuto mientras tanto...");
```

Orden real de ejecución:

1. "Solicitando datos..."
2. "Yo me ejecuto mientras tanto..."
3. (2 segundos después) → "Datos recibidos: { nombre: 'Ana', edad: 25 }"

Le decimos a obtenerDatos: cuando termines, ejecuta esta función con los datos que consigas.

7.4.1.2. Promesas

En programación, al igual que en la vida real, una promesa es algo que esperamos que se cumpla en el futuro.

Sin embargo, a medida que transcurre el tiempo, pueden suceder varias cosas:

- La promesa **se cumple** (promesa **aceptada**)
- La promesa **no se cumple** (promesa **rechazada**)
- La promesa se queda en un **estado incierto** (promesa **pendiente**)

Las promesas son como **una condición que queda en espera** y se resolverá en el futuro.

De esta manera, **nos permiten manejar código asíncrono sin bloquear la ejecución del resto del programa.**

Son un **Object** de tipo **Promise**.

Para crear nuestra propia promesa en una función necesitaremos hacer: **new Promise(resolve, reject)**

- **resolve** es una función callback que se usará cuando se cumpla la promesa
- **reject** es una función callback que se usará cuando se rechace la promesa

```
// 1 Creamos la Promesa
```

```
const promesa = new Promise((resolve, reject) => {  
  console.log("Promesa creada (pendiente)");
```

```
  
  // simulamos tarea asíncrona con setTimeout
```

```
  setTimeout(() => {
```

```
    const exito = Math.random() > 0.5; // decide aleatoriamente si es éxito o fallo
```

```
    if (exito) {
```

```
      resolve("¡La promesa se resolvió!");
```

```
      console.log("resolve() llamado dentro del setTimeout");
```

```
    } else {
```

```
      reject("La promesa falló 😬");
```

```
      console.log("reject() llamado dentro del setTimeout");
```

```
    }
```

```
  }, 5000); // 5 segundos de simulación
```

```
});
```

```
// 2 Mostramos la Promesa inmediatamente
```

```
console.log("Estado inicial de la Promesa:", promesa);
```

```
  
console.log("El código sigue ejecutándose mientras la Promesa espera ser resuelta o rechazada");
```

Promesa creada (pendiente)

Estado inicial de la Promesa: ▶ `Promise { <state>: "pending" }`

El código sigue ejecutándose mientras la Promesa espera ser resuelta o rechazada

▶ `GET http://127.0.0.1:5500/favicon.ico`

`reject()` llamado dentro del `setTimeout`

! ▶ `Uncaught (in promise) La promesa falló` 😞

Promesa creada (pendiente)

Estado inicial de la Promesa: ▶ `Promise { <state>: "pending" }`

El código sigue ejecutándose mientras la Promesa espera ser resuelta o rechazada

▶ `GET http://127.0.0.1:5500/favicon.ico`

`resolve()` llamado dentro del `setTimeout`

- Llamar a `resolve("¡La promesa se resolvió!")` no imprime automáticamente ese mensaje.
- `resolve()` solo cambia el estado interno de la Promesa a `fulfilled` y almacena el valor `"¡La promesa se resolvió!"`.
- Para "ver" ese valor, necesitas consumirlo con `.then()` o `await`.

Otra opción es consultar el valor de la promesa pasados unos segundos.

```
// Inspeccionamos la Promesa después de 7 segundos
setTimeout(() => {
  console.log("Estado de la Promesa después de 7s:", promesa);
}, 7000);
```

Promesa creada (pendiente)

Estado inicial de la Promesa: ▶ `Promise { <state>: "pending" }`

El código sigue ejecutándose mientras la Promesa espera ser resuelta o rechazada

▶ `GET http://127.0.0.1:5500/favicon.ico`

`resolve()` llamado dentro del `setTimeout`

Estado de la Promesa después de 7s: ▶ `Promise { <state>: "fulfilled", <value>: "¡La promesa se resolvió!" }`

Podemos trabajar con **thennables** donde se utiliza un método llamado **.then()**:

- **.then(function resolve)**: Ejecuta la función cuando la promesa **se acepta**.
- **.catch(function reject)**: Ejecuta la función cuando la promesa **se rechaza**.
- **.then(function resolve, function reject)**: **.then()** + **.catch()**.
- **.finally(function end)**: Ejecuta la función cuando **sale de pendiente**.

Añadimos este código a nuestro ejemplo de promesa ...

```
// 2 Consumimos el valor resuelto con .then
promesa.then(function(valor) {
  console.log("Valor recibido en .then:", valor);
});

// 3 También podemos capturar el error con .catch
promesa.catch(function(error) {
  console.log("Error recibido en .catch:", error);
});
```

reject() llamado dentro del setTimeout

Error recibido en .catch: La promesa falló 😬

resolve() llamado dentro del setTimeout

Valor recibido en .then: ¡La promesa se resolvió!

○ bien este ...

```
// 2 Usamos .then con dos funciones: resolve y reject
promesa.then(
  function(valor) { // función para resolve
    console.log("Valor recibido (resolve):", valor);
  },
  function(error) { // función para reject
    console.log("Error recibido (reject):", error);
  }
);
```

reject() llamado dentro del setTimeout

Error recibido (reject): La promesa falló 😬

resolve() llamado dentro del setTimeout

Valor recibido (resolve): ¡La promesa se resolvió!

Y finalmente ...

```
// 3 .finally siempre se ejecuta
promesa.finally(function() {
|   console.log("Fin de la Promesa (finally): ¡Esto siempre se ejecuta!");
| });
```

Promesa creada (pendiente)

El código sigue ejecutándose mientras la Promesa está pendiente

Live reload enabled.

resolve() llamado dentro del setTimeout

Valor recibido (resolve): ¡La promesa se resolvió!

Fin de la Promesa (finally): ¡Esto siempre se ejecuta!

Podemos encadenar promesas ...

```
* sumaPromesa: genera una nueva promesa que realiza la suma
* de dos números pasados por parámetro a los dos segundos
*/
function sumaPromesa(n1, n2) {
  return new Promise((resolve, reject) => {
    // siempre será aceptada; por lo que en función de lo que reciba, realizará un operación u otra
    setTimeout(() => resolve(n1 + n2), 2000);
  });
}
```

Esta es la última promesa

28

Terminado.

```
/**
 * Consumo con 'thennables':
 * Se pueden encadenar promesas: Sumar 4 + 8 + 6 + 10
 */

sumaPromesa(4,8) // primera
.then(r => sumaPromesa(r, 6)) // segunda
.then(r => sumaPromesa(r, 10).finally(() => console.log("Esta es la última promesa"))) // tercera
.then(r => console.log(r))
.catch(error => console.error('Error: ' + error))
.finally(() => console.log("Terminado."));
```

Tiempo →

0s

2s

4s

6s

|

|

|

|

└ "Esta es la última promesa"

└ 28

└ "Terminado."

└ sumaPromesa(12,6) → 18

└ sumaPromesa(4,8) → 12

Inicio

Ideas clave →

- Las promesas NO se ejecutan en paralelo
- Cada `.then()` :
 - espera al anterior
 - recibe su resultado
- El tiempo total = $2s \times \text{número de promesas}$
- `finally()` no altera el valor, solo ejecuta código

7.4.1.3. **async/await**

Las instrucciones **async** y **await** fueron introducidas en el estándar ES2017.

Se utilizan como sintaxis **alternativa de utilizar promesas** mucho más simple y clara de escribir.

- **await** permite “pausar” la ejecución dentro de una función **async** hasta que una Promesa se resuelva.
- No bloquea toda la aplicación; el resto del código sigue ejecutándose.
- **await** hace que el código asíncrono sea más fácil de leer, y solo se puede usar dentro de funciones **async**.
- Una función **async** siempre devuelve una Promesa.

```
// 1 Creamos una Promesa que tarda 5 segundos en resolverse
```

```
const promesa = new Promise((resolve) => {  
  console.log("Promesa creada (pendiente)");  
  setTimeout(() => {  
    resolve("¡Promesa resuelta!");  
    console.log("resolve() llamado dentro de la Promesa");  
  }, 5000);  
});
```

```
// 2 Función async que usa await
```

```
async function ejemplo() {  
  console.log("Función ejemplo: antes de await");  
  
  const valor = await promesa; // aquí "esperamos" la Promesa  
  console.log("Función ejemplo: después de await → valor:", valor);  
  
  console.log("Función ejemplo: fin de la función async");  
}
```

```
// 3 Llamamos a la función
```

```
ejemplo();
```

```
console.log("Código fuera de la función async sigue ejecutándose normalmente");
```

Promesa creada (pendiente)

Función ejemplo: antes de await

Código fuera de la función async sigue ejecutándose normalmente

Live reload enabled.

resolve() llamado dentro de la Promesa

Función ejemplo: después de await → valor: ¡Promesa resuelta!

Función ejemplo: fin de la función async

```
// 1 Creamos una "Promesa de datos" que tarda 5 segundos en resolverse
```

```
const obtenerUsuarios = new Promise((resolve) => {  
  console.log("Promesa creada: obteniendo usuarios...");
```

```
  setTimeout(() => {
```

```
    const usuarios = ["Ana", "Luis", "Carlos", "María"]; // datos claros
```

```
    resolve(usuarios);
```

```
    console.log("resolve() llamado: usuarios listos");
```

```
  }, 5000);
```

```
});
```

```
// 2 Función async que usa await para "esperar" la Promesa
```

```
async function mostrarUsuarios() {
```

```
  console.log("Función mostrarUsuarios: antes de await");
```

```
  const lista = await obtenerUsuarios; // espera que la Promesa se resuelva
```

```
  console.log("Función mostrarUsuarios: después de await → lista de usuarios:", lista);
```

```
  console.log("Función mostrarUsuarios: fin de la función async");
```

```
}
```

```
// 3 Llamamos a la función
```

```
mostrarUsuarios();
```

```
console.log("Código fuera de la función async sigue ejecutándose normalmente");
```

Promesa creada: obteniendo usuarios...

Función mostrarUsuarios: antes de await

Código fuera de la función async sigue ejecutándose normalmente

Live reload enabled.

resolve() llamado: usuarios listos

Función mostrarUsuarios: después de await → lista de usuarios: ▶ [Array\(4\)](#)

Función mostrarUsuarios: fin de la función async

Podemos usar “varios awaits dentro un sync” ...

Tenemos la función **sumaPromesa** ...

```
// Función que devuelve una Promesa y suma dos números después de 2 segundos
function sumaPromesa(n1, n2) {
  return new Promise((resolve, reject) => {
    console.log(`Iniciando suma de ${n1} + ${n2}...`);
    setTimeout(() => {
      const resultado = n1 + n2;
      console.log(`Resultado de ${n1} + ${n2} calculado: ${resultado}`);
      resolve(resultado);
    }, 2000);
  });
}
```

... y la llamamos con varios "awaits" ...

```
/* Función asíncrona para sumar 4 + 8 + 6 + 10 usando async/await */
async function sumarAsync() {
  try {
    console.log("Inicio de sumarAsync");

    const resultado1 = await sumaPromesa(4, 8);    // 4 + 8 = 12
    const resultado2 = await sumaPromesa(resultado1, 6); // 12 + 6 = 18
    const resultadoFinal = await sumaPromesa(resultado2, 10); // 18 + 10 = 28

    console.log("Todas las sumas terminadas");
    console.log("Resultado final:", resultadoFinal); // 28
  } catch (error) {
    console.error('Error:', error);
  } finally {
    console.log("Terminado sumarAsync");
  }
}
```

```
// 1 Llamamos a la función asíncrona  
sumarAsync();  
  
// 2 Mostramos que el hilo principal sigue ejecutándose  
console.log("Este mensaje se imprime mientras se esperan las sumas");
```

Inicio de sumarAsync

Iniciando suma de 4 + 8...

Este mensaje se imprime mientras se esperan las sumas

Live reload enabled.

Resultado de 4 + 8 calculado: 12

Iniciando suma de 12 + 6...

Resultado de 12 + 6 calculado: 18

Iniciando suma de 18 + 10...

Resultado de 18 + 10 calculado: 28

Todas las sumas terminadas

Resultado final: 28

Terminado sumarAsync

Uso de **async/await** + **.then()**

Función **async** → siempre devuelve una Promesa

Para obtener el valor de la Promesa fuera de la función **async**, hay dos formas:

- **.then**(valor => ...)
- **await** dentro de otra función **async**

Sin **.then** o **await**, solo tendríamos la Promesa, no el valor resuelto.

```
function resolveAfter2Seconds(x) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(x);  
    }, 2000);  
  });  
}
```

► Promise { <state>: "pending" }

▼ Promise { <state>: "pending" }

<state>: "fulfilled"

<value>: 60

► <prototype>: Promise.prototype { ... }

```
async function add1(x) {  
  const a = await resolveAfter2Seconds(20); // 'bloquea'  
  const b = await resolveAfter2Seconds(30); // 'bloquea'  
  return x + a + b; // resuelve  
}
```

```
const resultado = add1(10);  
console.log(resultado);
```

```
function resolveAfter2Seconds(x) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(x);  
    }, 2000);  
  });  
}
```

```
async function add1(x) {  
  const a = await resolveAfter2Seconds(20); // 'bloquea'  
  const b = await resolveAfter2Seconds(30); // 'bloquea'  
  return x + a + b; // resuelve  
}
```

```
const resultado = add1(10);  
console.log(resultado);
```

```
setTimeout(() => {  
  console.log("Estado de la Promesa después de 7s:", resultado);  
}, 7000);
```

► Promise { <state>: "pending" }

Live reload enabled.

Estado de la Promesa después de 7s: ► Promise { <state>: "fulfilled", <value>: 60 }

```
function resolveAfter2Seconds(x) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(x);  
    }, 2000);  
  });  
}  
  
async function add1(x) {  
  const a = await resolveAfter2Seconds(20); // 'bloquea'  
  const b = await resolveAfter2Seconds(30); // 'bloquea'  
  return x + a + b; // resuelve  
}  
  
add1(10).then((v) => {  
  console.log("Add1: " + v); // prints 60 after 4 seconds.  
});
```

Add1: 60

Concurrencia de promesas

```
function resolveAfter2Seconds(x) {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(x);  
    }, 2000);  
  });  
}
```

Las promesas empiezan a ejecutarse cuando se crean, así que el **setTimeout** de las dos empieza al mismo tiempo.

```
async function add2(x) {  
  const p_a = resolveAfter2Seconds(20);  
  const p_b = resolveAfter2Seconds(30);  
  return x + (await p_a) + (await p_b);  
}  
  
add2(10).then((v) => {  
  console.log("Add2: " + v); // prints 60 after 2 seconds.  
});
```

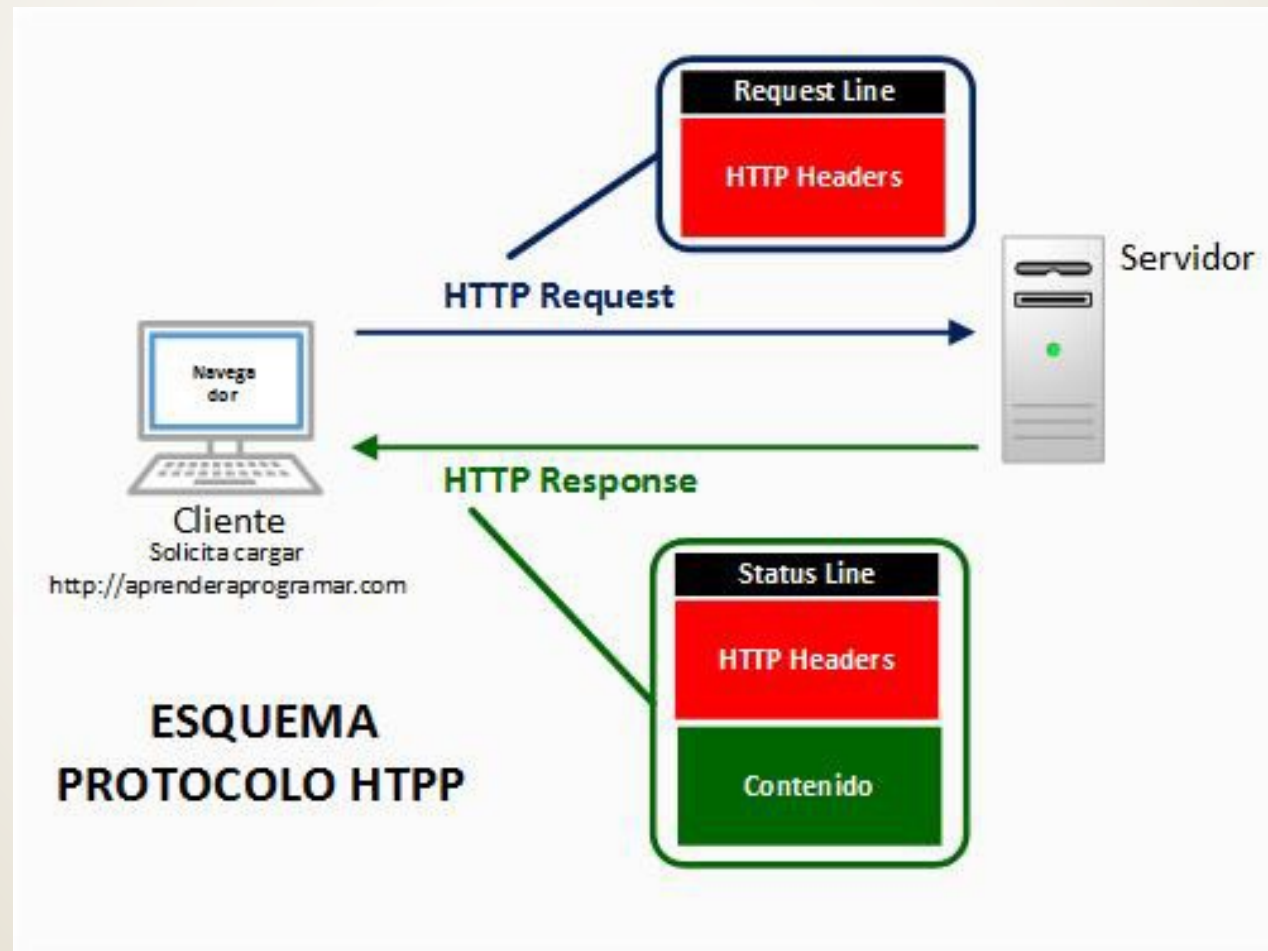
7.4.2. Peticiones HTTP

Una petición HTTP es la acción de **solicitar a un servidor web un archivo** (html, imagen, js, etc.).

A través de la petición, el navegador puede descargar el archivo para mostrarlo.

La petición HTTP se realiza desde JavaScript **de forma transparente para el usuario**, permitiendo descargar y procesar la información sin mostrarla directamente en la página.

Por lo que, permite realizar **actualizaciones parciales** del contenido, actualizando solo una sección específica de la página en tiempo real **sin necesidad de recargar la página por completo**.



Formas de realizar peticiones HTTP mediante AJAX:

- Nativos del navegador:

- **XMLHttpRequest(XHR)**: El más antiguo y verboso (más código).
- **Fetch**: Sistema nativo de peticiones basado en promesas.

- Librerías externas:

- Axios, superagent o **frisbee** (basada en fetch y utilizada normalmente en React)

7.4.2.1. XMLHttpRequest (XHR)

Objeto especial de Javascript que permite **realizar peticiones HTTP asíncronas** (AJAX) de forma nativa desde Javascript.

Es la primera implementación que existió, antes de que surgiera **fetch**, el estándar recomendado actualmente.

Es un formato que se vuelve un poco más complejo a medida que vamos realizando comprobaciones, ya que se realiza todo más a bajo nivel.

```
// Petición GET con XHR
function getProductos() {
    // 1. Creamos la instancia del objeto
    const xhr = new XMLHttpRequest();
    // 2. Establecemos la comunicación con el servidor
    // Método GET:
    xhr.open('GET', `${SERVER}/productos`, true);
    // 3. Enviamos la petición al servidor
    xhr.send();
    // 4. Ponemos una escucha al objeto
    xhr.onreadystatechange = function () {
        if (xhr.readyState === 4) {
            if (xhr.status === 200) {
                const json = JSON.parse(xhr.responseText); // JSON a Objeto -- CALLBACK
                console.log(json);
            } else {
                console.error("Fallo en la obtención de productos:", xhr.statusText);
            }
        }
    };
}
```

```
// Insertar producto con XHR
function postProducto(producto) {
    // 1. Creamos la instancia del objeto
    const xhr = new XMLHttpRequest();
    // 2. Establecemos la comunicación con el servidor
    //    -- Añadimos cabeceras a la petición
    //    Método POST:
    xhr.open('POST', `${SERVER}/productos`, true);
    xhr.setRequestHeader('Content-Type', 'application/json'); // indicamos que enviamos un json
    // 3. Enviamos la petición al servidor
    xhr.send(JSON.stringify(producto)); // pasamos producto de Objeto a JSON
    // 4. Ponemos una escucha al objeto
    xhr.onreadystatechange = function () {
        if (xhr.readyState === 4) {
            if (xhr.status === 200 || xhr.status === 201) {
                const json = JSON.parse(xhr.responseText); // JSON a Objeto -- CALLBACK
                console.log(json);
            } else {
                console.error("Fallo insertando el producto:", xhr.statusText);
            }
        }
    };
}
```

7.4.2.2. Fetch

La **API Fetch** permite realizar **peticiones AJAX basándose en promesas**, a diferencia de XMLHttpRequest, que es una API basada en Callbacks; por lo que, con fetch() podemos evitar el “infierno de las devoluciones de llamadas”.

El método **fetch()** devuelve un **objeto Promise** que representa la respuesta a la petición, independientemente de si se realiza correctamente o no.

```
// Petición GET (por defecto) con Fetch
function getProductos() {
  fetch(`${SERVER}/productos`)
    .then(resp => {
      // si el status no está entre 200 y 299, se produce error
      if (!resp.ok) throw new Error(`Error: ${resp.status} ${resp.statusText}`);
      return resp.json(); // convierte JSON a Objeto
    })
    .then(json => {
      console.log(json);
      productos = json.data;
      replaceProductos();
    })
    .catch(error => {
      console.error("Fallo en la obtención de productos:", error);
    });
}
```

fetch(URL)

- cuando llegue la respuesta, léela como JSON
- cuando el JSON esté listo, úsalo

Es importante destacar que **fetch()** no rechaza una Promesa en el caso de códigos de estado de error HTTP (como 404 o 500), como lo hace XHR.

Es por ello que se deberá verificar la **propiedad ok** del objeto de respuesta para determinar si la petición fue exitosa.

Referencias ...



<https://developer.mozilla.org/es/docs/Web/API/XMLHttpRequest>

https://developer.mozilla.org/es/docs/Web/API/Fetch_API

Promesas en grupo

En ocasiones se necesita realizar **más de una petición** al servidor y renderizar la información **a la vez** , pudiendo ser dependiente una de la otra.

La solución pasa por utilizar el método **Promise.all()**.

Más info:



<https://lenguajejs.com/asincronia/control/promise-all/>

Ejemplos de módulos y promesas

→ importación dinámica de módulos utilizando promesas

<https://github.com/mdn/js-examples/blob/main/module-examples/dynamic-module-imports/main.js>

→ importación dinámica de módulos top level await
(contexto de utilización de await sin async - módulo
getColor.js)

<https://github.com/mdn/js-examples/tree/main/module-examples/top-level-await>